COMPUTER SCIENCE TRIPOS Part IB – 2022 – Paper 5

8 Introduction to Computer Architecture (swm11)

Technology trends and design challenges, system-on-chip design

(*a*) Why are modern systems-on-chip (SoC) heterogeneous (i.e. contain a range of different processor cores)? [4 marks]

Answer: Different cores trade-off power consumption and performance. Typically smaller cores are much more power efficient than big cores but big cores have higher performance. A heterogeneous system allows the operating system or other run-time to choose what work gets scheduled on which core in order to optimise power and performance.

Technology trends and design challenges, system-on-chip design

(*b*) What is the *von Neumann bottleneck* and to what extent do modern SoCs suffer from it? [4 marks]

Answer: The von Neumann bottleneck was coined by John Backus and stems from John von Neumann's thought experiment that all processor state could be stored in one linear memory (see the *First Draft Report on EDVAC* published in 1945) with the bus between the processor and memory limiting throughput. Today we use a complex memory hierarchy on SoCs but often refer to the external memory bandwidth as the von Neumann bottleneck. With so many processor cores and accelerators, there is enormous contention for memory bandwidth. While caches help, ultimately the external memory interface is the bottleneck due to limits on the number of pins on the chip and the frequency they can operate at.

Note that the historical context is beyond what I would expect in an answer scoring full marks.

Support for operating systems

(*c*) How is virtual memory used to provide isolation between applications? [4 marks]

Answer: Applications running on conventional application-class CPUs use virtual addressing for memory allocation and protection/isolation. Applications undertake memory operations in virtual address space and the hardware translates virtual addresses to physical ones using page tables (cached using TLBs) before accessing memory. Page tables are typically per application, so if a translation is not in the page table for that application it cannot access that virtual address. Page tables also contain permission information (read, write, execute, etc.). Full credit can be achieved by pointing out that everyday techniques work on a SoC.

Full credit can be awarded for just the points above.

IOMMUs (input/output memory management units) can be used to preserve the virtual memory model across the SoC though use of this technology is still being improved. No IOMMU or poor use of IOMMUs opens up security holes. Security holes can also be opened through the OS incorrectly modifying page tables due to a bug or other vulnerability.

Graphics processing units

(*d*) How do GPUs hide memory access latency? [4 marks]

Answer: GPUs use multithreading (warps) to hide memory access latency. When one warp blocks waiting for memory, other warps not waiting for memory can be scheduled.

Pipelining, Graphics processing units

(*e*) Why is conditional program flow control managed differently on GPUs and CPUs? [4 marks]

Answer: On a conventional CPU, conditional branches cause the program counter to be updated, thereby changing the thread of control to a new sequence of instructions. GPUs use a single instruction stream for many threads, so most conditional branches are replaced with conditional or *predicated* execution. Instructions are issued to all thread execution pipelines together with a predicate that indicates if the instruction should be executed by that thread or simply ignored.